Diffusion Forcing: Next-token Prediction Meets Full-Sequence Diffusion

NeurIPS 2024 Paper Accepted

麻省理工学院计算机科学与人工智能实验室

Boyuan Chen
MIT CSAIL
boyuanc@mit.edu

Diego Marti Monso*
Technical University of Munich
diego.marti@tum.de

Yilun Du
MIT CSAIL
yilundu@mit.edu

Max Simchowitz
MIT CSAIL
msimchow@mit.edu

Russ Tedrake
MIT CSAIL
russt@mit.edu

Vincent Sitzmann
MIT CSAIL
sitzmann@mit.edu

Video Generation Methods

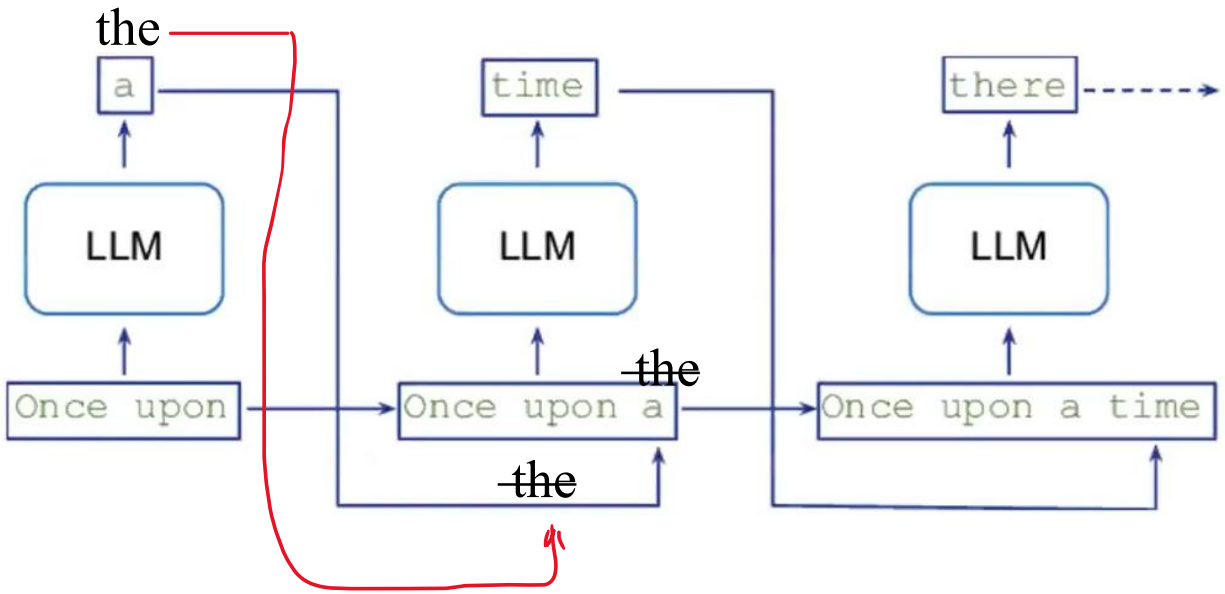
- Next-Token（AR）
- Full Seq Diffusion

Preliminaries

Autorregressive Model (AR)： predict the next frame x_{t+1} given past frames $x_{1:t}$

- Training：

"Teacher Forcing": predicts the immediate next token based on previous tokens



- Strength

  -Variable length generation

- Weaknesses

  - AR cannot add new optimization objectives in inference time（路径规划任务） $\nabla_x \log(y|x)$

  - quality degradation problem caused by the mismatch of training and inference samples

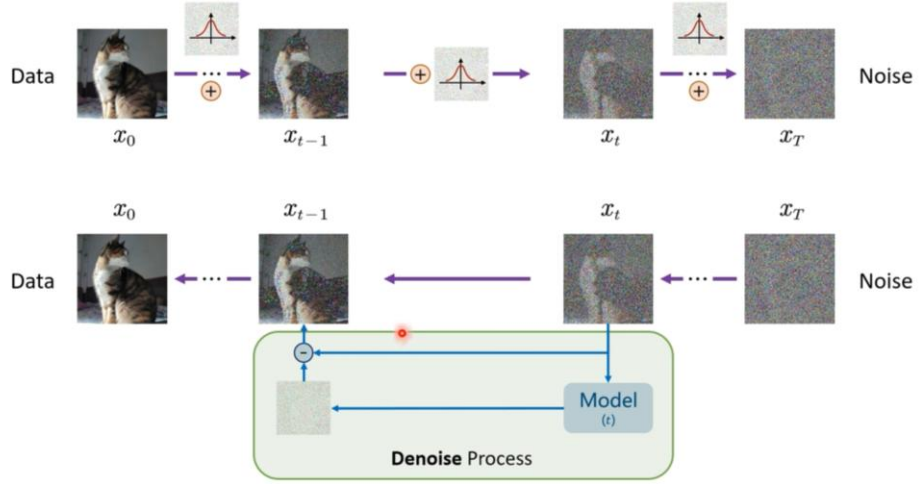
  - certainty

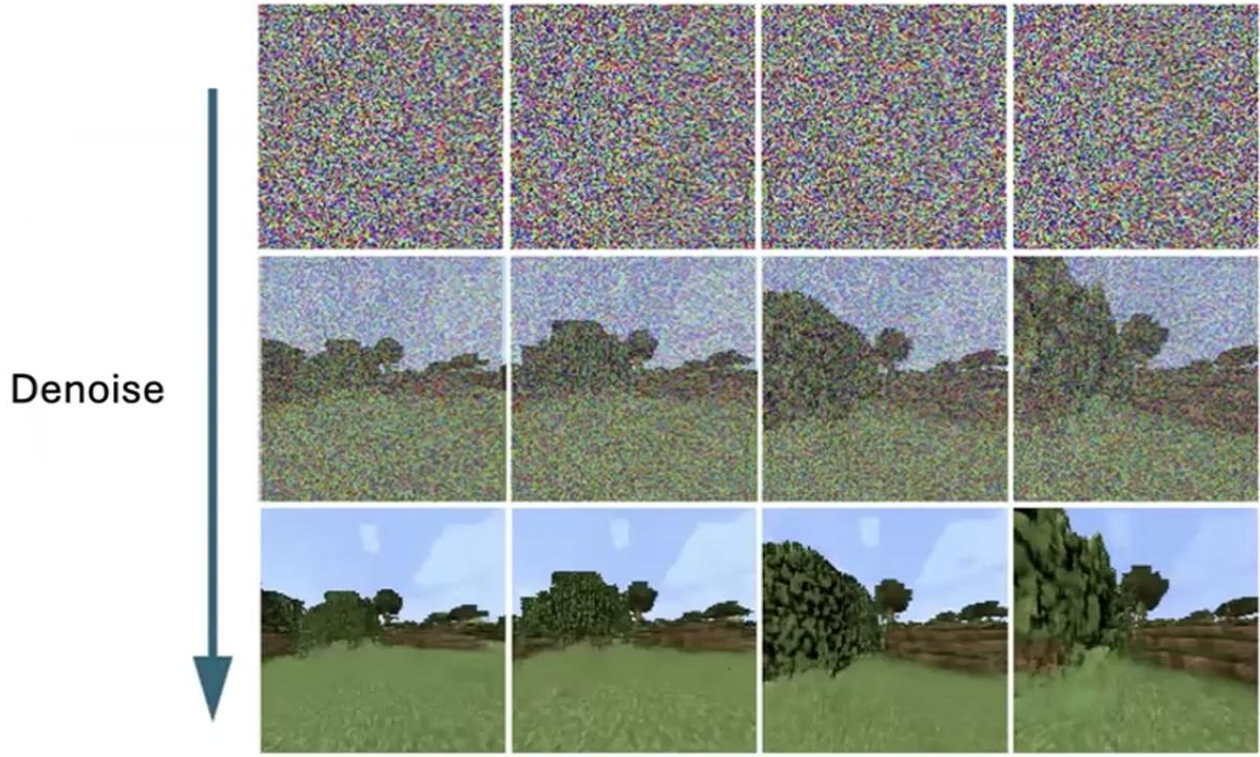
Preliminaries

Diffusion Model (Full Sequence)

- Weaknesses
  - **Fixed sequenced length**, has to combine AR to generate long Video

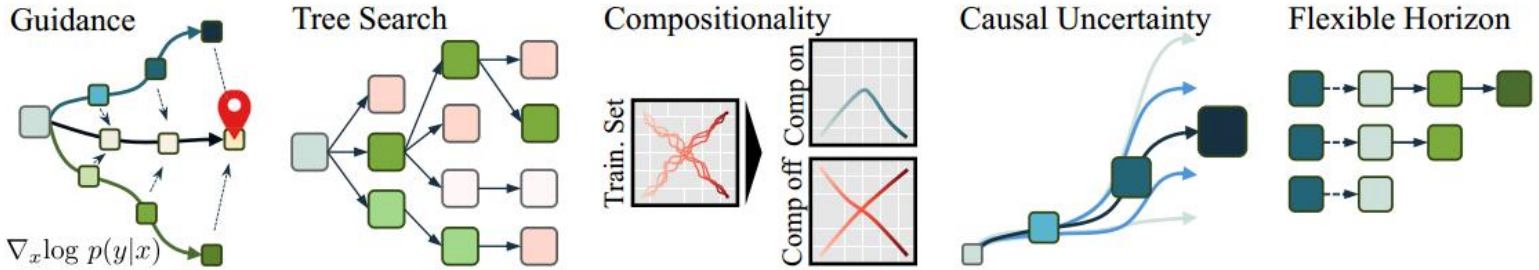
- Strength
  - Can add **Classifier-guidance** in inference time
  - there is only a little degradation within training sequence length.





- past
- future

Diffusion Forcing



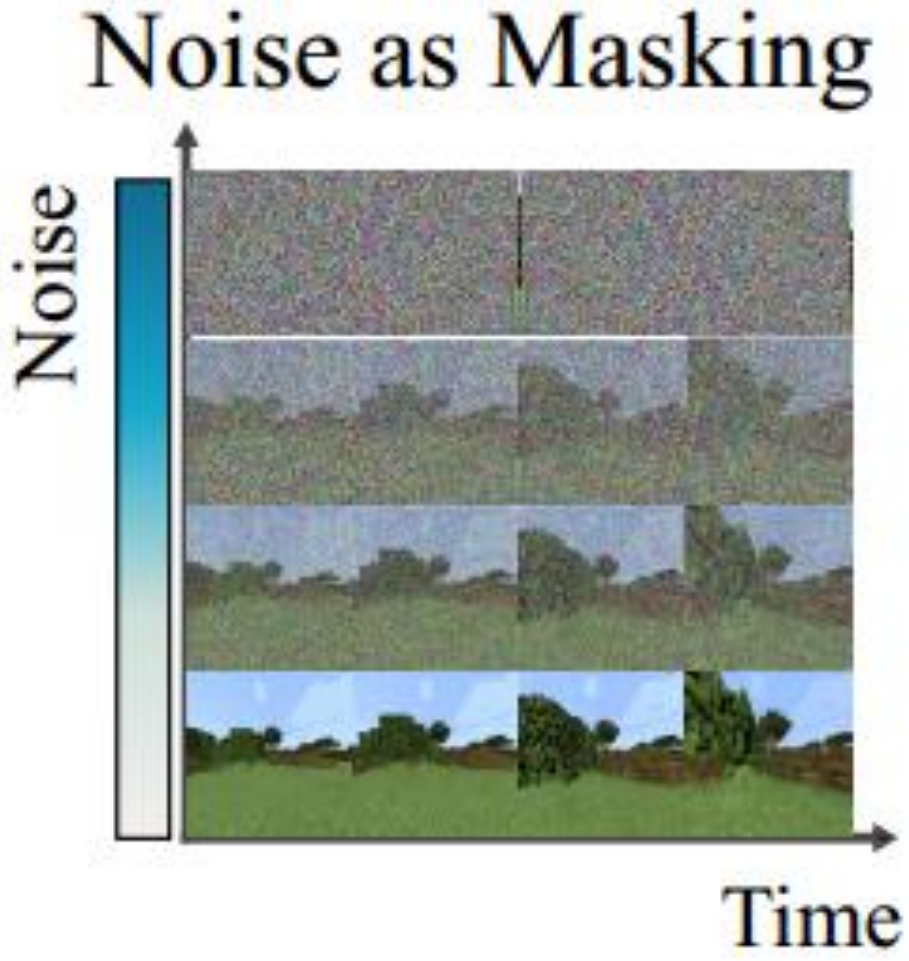
	Guidance	Tree Search	Compositionality	Causal Uncertainty	Flexible Horizon
Teacher Forcing	✗	✓	✓	✓	✓
Full-Seq. Diffusion	✓	✗	✗	✗	✗
Diffusion Forcing	✓	✓	✓	✓	✓

Figure 1: **Diffusion Forcing capabilities.** Today, different applications such as language modeling [6], planning [37], or video generation [32, 70] rely on *either* auto-regressive next-token prediction *or* full-sequence diffusion, according to their respective unique capabilities. The proposed Diffusion Forcing is a novel sequence generative model that enjoys key strengths of both model types.

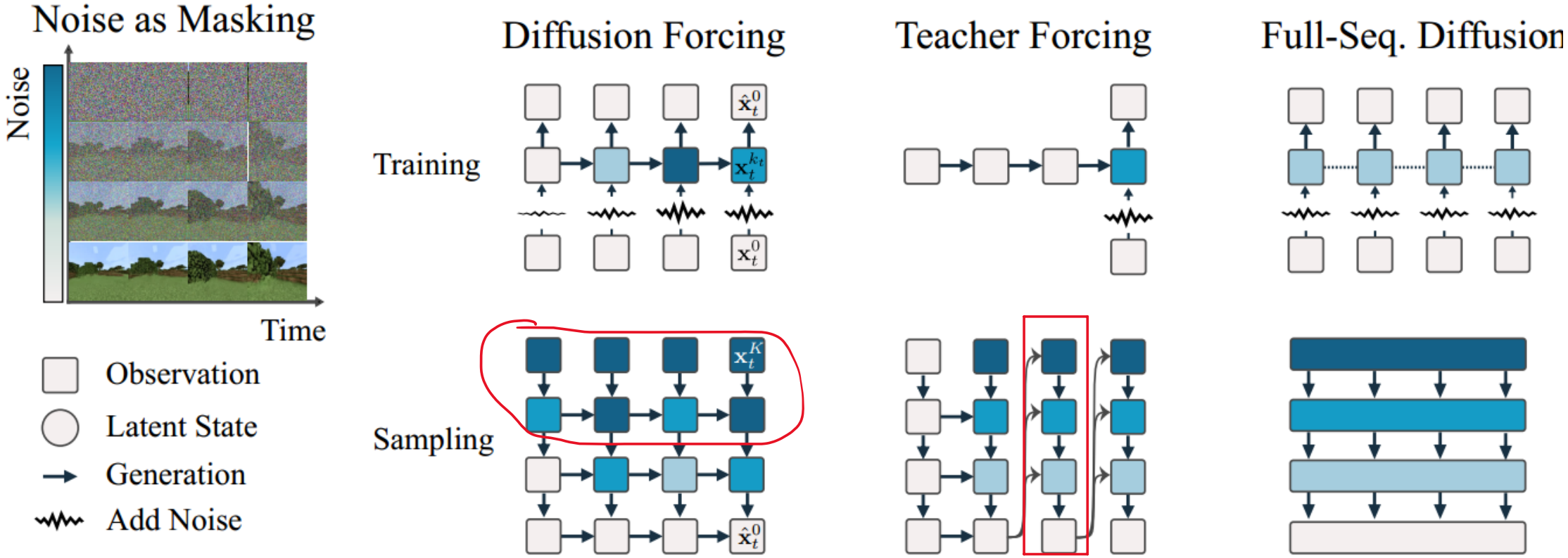
Masking Strategy——Noise

Masking Strategy

- Full noise –no information
- Less Noise - uncertainty



Diffusion Forcing



Masking Strategy—— Independent Noise

- Earlier elements : Less Noise
- Newer elements : Large Noise

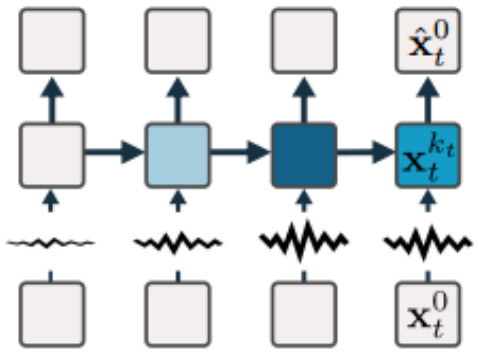
——Future uncertainty

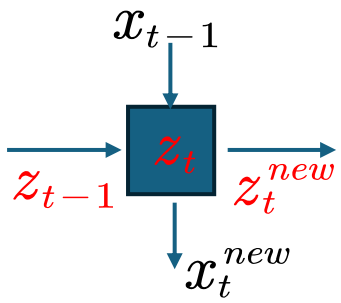
All the elements of different noise levels are denoised at the same time in diffusion models.

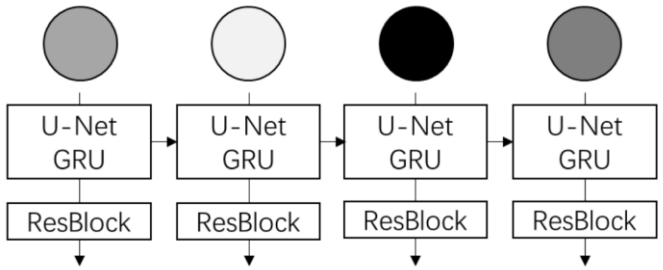
一次性生成视野内的全部结果，而非teacher forcing一次仅生成一个确定结果

Diffusion Forcing

Training



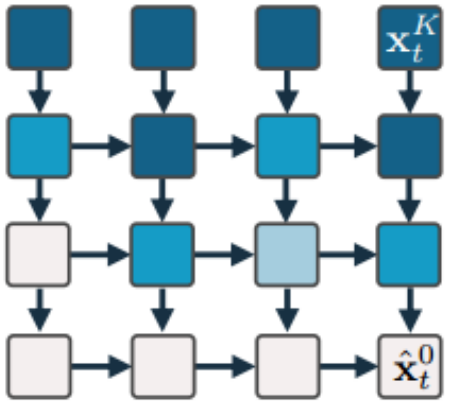




U-Net+GRU

模型单元具有相同input output特性、因果特性、无序列长度限制

Sampling



训练策略:

每个噪声大小独立且随机组合,而非训练中就使用（低噪声-高噪声）的噪声策略

——在“任意噪声历史 + 任意噪声当前观测”下做预测。

- RNN: dynamics model —— z_t :latant feature $p_\theta(\mathbf{z}_t|\mathbf{z}_{t-1}, \mathbf{x}_t^{k_t}, k_t)$
- Diffusion: observation model——DDIM/DDPM $p_\theta(\mathbf{x}_t^0|\mathbf{z}_t)$

Masking Strategy—— Independent Noise

- Earlier elements : Less Noise
- Newer elements : Large Noise

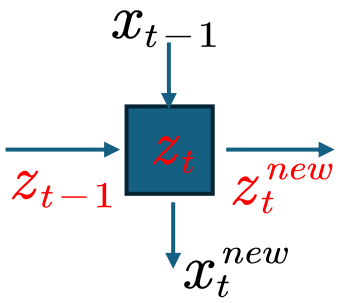
——Future uncertainty

- Objective:

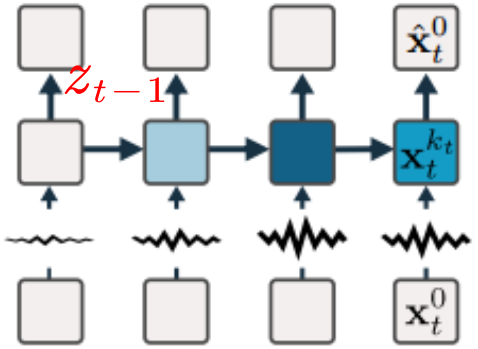
$$\mathbb{E}_{\substack{k_t, \mathbf{x}_t, \boldsymbol{\epsilon}_t \\ \mathbf{z}_t \sim p_\theta(\mathbf{z}_t|\mathbf{z}_{t-1}, \mathbf{x}_t^{k_t}, k_t)}} \sum_{t=1}^{T} \left[\|\boldsymbol{\epsilon}_t - \boldsymbol{\epsilon}_\theta(\mathbf{z}_{t-1}, \mathbf{x}_t^{k_t}, k_t)\|^2 \right], \quad k_t = K$$

$$\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\, \boldsymbol{x}_0 + \sqrt{1 - \bar{\alpha}_t}\, \boldsymbol{\epsilon}_t$$

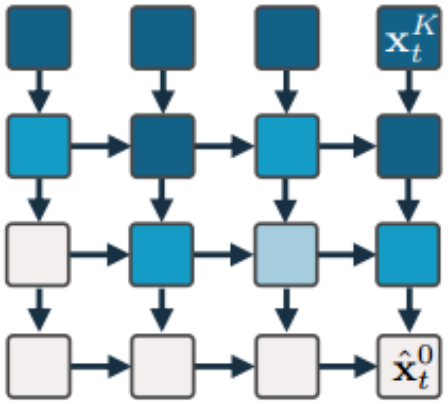
Diffusion Forcing——Training



Training



Sampling



Algorithm 1 Diffusion Forcing Training

loop
  Sample tajectory of observations $(\mathbf{x}_1, ..., \mathbf{x}_T)$.
  for $t = 1, ..., T$ **do**
    Sample independent noise level $k_t \in \{0, 1, ..., K\}$
    $\mathbf{x}_t^{k_t} = \text{ForwardDiffuse}(\mathbf{x}_t, k_t)$
    Define $\epsilon_t = \frac{\mathbf{x}_t^{k_t} - \sqrt{\bar{\alpha}_{k_t}}\mathbf{x}_t}{\sqrt{1-\bar{\alpha}_{k_t}}}$
    Update $\mathbf{z}_t \sim p_\theta(\mathbf{z}_t | \mathbf{z}_{t-1}, \mathbf{x}_t^{k_t}, k_t)$.
    Set $\hat{\epsilon}_t = \epsilon_\theta(\mathbf{z}_{t-1}, \mathbf{x}_t^{k_t}, k_t)$
  end for
  $L = \text{MSELoss}([\hat{\epsilon}_1, ..., \hat{\epsilon}_n], [\epsilon_1, ..., \epsilon_n])$
  Backprop with L and update θ
end loop

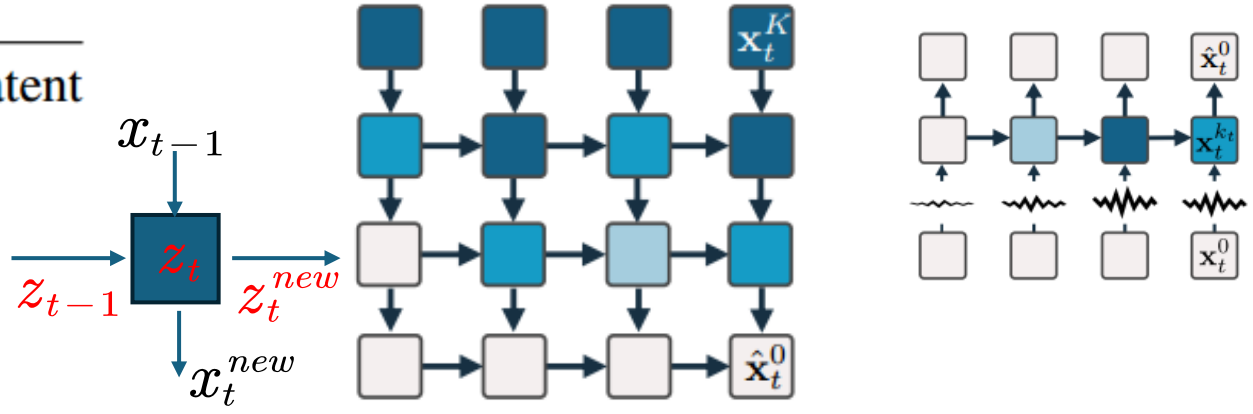
DDPM的训练过程：

——预测噪声

Diffusion Forcing——Sampling

Algorithm 2 DF Sampling with Guidance

Input: Model θ, scheduling matrix $\mathcal{K}$, initial latent $\mathbf{z}_0$, guidance cost $c(\cdot)$.
Initialize $\mathbf{x}_1, \ldots, \mathbf{x}_T \sim \mathcal{N}(0, \sigma_K^2 I)$.
for row $m = M-1, \ldots, 0$ **do**
  for $t = 1, \ldots, T$ **do**
    $\mathbf{z}_t^{\text{new}} \sim p_\theta(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_t, \mathcal{K}_{m+1,t})$.
    $k \leftarrow \mathcal{K}_{m,t}$, $\mathbf{w} \sim \mathcal{N}(0, \mathbf{I})$.
    $\mathbf{x}_t^{\text{new}} \leftarrow \frac{1}{\sqrt{\alpha_k}}(\mathbf{x}_t - \frac{1-\alpha_k}{\sqrt{1-\bar{\alpha}_k}}\epsilon_\theta(\mathbf{z}_t^{\text{new}}, \mathbf{x}_t, k)) + \sigma_k \mathbf{w}$
    Update $\mathbf{z}_t \leftarrow \mathbf{z}_t^{\text{new}}$.
  end for
  $\mathbf{x}_{1:H} \leftarrow$AddGuidance$(\mathbf{x}_{1:H}^{\text{new}}, \nabla_{\mathbf{x}} \log c(\mathbf{x}_{1:H}^{\text{new}}))$
end for
Return $\mathbf{x}_{1:T}$.



$$\mathcal{K}^{\text{pyramid}} = \begin{bmatrix} K & K & K & \ldots & K \\ K-1 & K & K & \ldots & K \\ K-2 & K-1 & K & \ldots & K \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & 2 & 3 & \ldots & H \\ 0 & 1 & 2 & \ldots & H-1 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \ldots & 1 \\ 0 & 0 & 0 & \ldots & 0 \end{bmatrix}$$

Denoising Timestep

Frame Index

sampling策略：

噪声等级矩阵K：扩散速度递减，保证未来不确定性（K代表当前噪声等级）

按行循环：一次性得到当前扩散时间步的全部结果

Connection to other sequence training schemes

通过定义K，实现不同生成模型

Autoregressive：

denoising

$$[\mathbf{x}_1^K, \mathbf{x}_2^K, \mathbf{x}_3^K]^\top$$
$$[\mathbf{x}_1^{K//2}, \mathbf{x}_2^K, \mathbf{x}_3^K]^\top$$
$$[\mathbf{x}_1^0, \mathbf{x}_2^K, \mathbf{x}_3^K]^\top,$$
$$[\mathbf{x}_1^0, \mathbf{x}_2^{K//2}, \mathbf{x}_3^K]^\top$$
$$[\mathbf{x}_1^0, \mathbf{x}_2^0, \mathbf{x}_3^K]^\top,$$
$$[\mathbf{x}_1^0, \mathbf{x}_2^0, \mathbf{x}_3^{K//2}]^\top,$$
$$[\mathbf{x}_1^0, \mathbf{x}_2^0, \mathbf{x}_3^0]^\top.$$

T

Full Seq：

$$[\mathbf{x}_1^K, \mathbf{x}_2^K, \mathbf{x}_3^K]^\top$$
$$[\mathbf{x}_1^{K//2}, \mathbf{x}_2^{K//2}, \mathbf{x}_3^{K//2}]^\top,$$
$$[\mathbf{x}_1^0, \mathbf{x}_2^0, \mathbf{x}_3^0]^\top.$$

Diffusion Forcing with causal uncertainty

$$[\mathbf{x}_1^K, \mathbf{x}_2^K, \mathbf{x}_3^K]^\top$$
$$[\mathbf{x}_1^{K//2}, \mathbf{x}_2^K, \mathbf{x}_3^K]^\top$$
$$[\mathbf{x}_1^0, \mathbf{x}_2^{K//2}, \mathbf{x}_3^K]^\top$$
$$[\mathbf{x}_1^0, \mathbf{x}_2^0, \mathbf{x}_3^{K//2}]^\top$$
$$[\mathbf{x}_1^0, \mathbf{x}_2^0, \mathbf{x}_3^0]^\top$$

Diffusion Forcing with stabilization:

生成出来的图片应该当作有噪声的ground truth

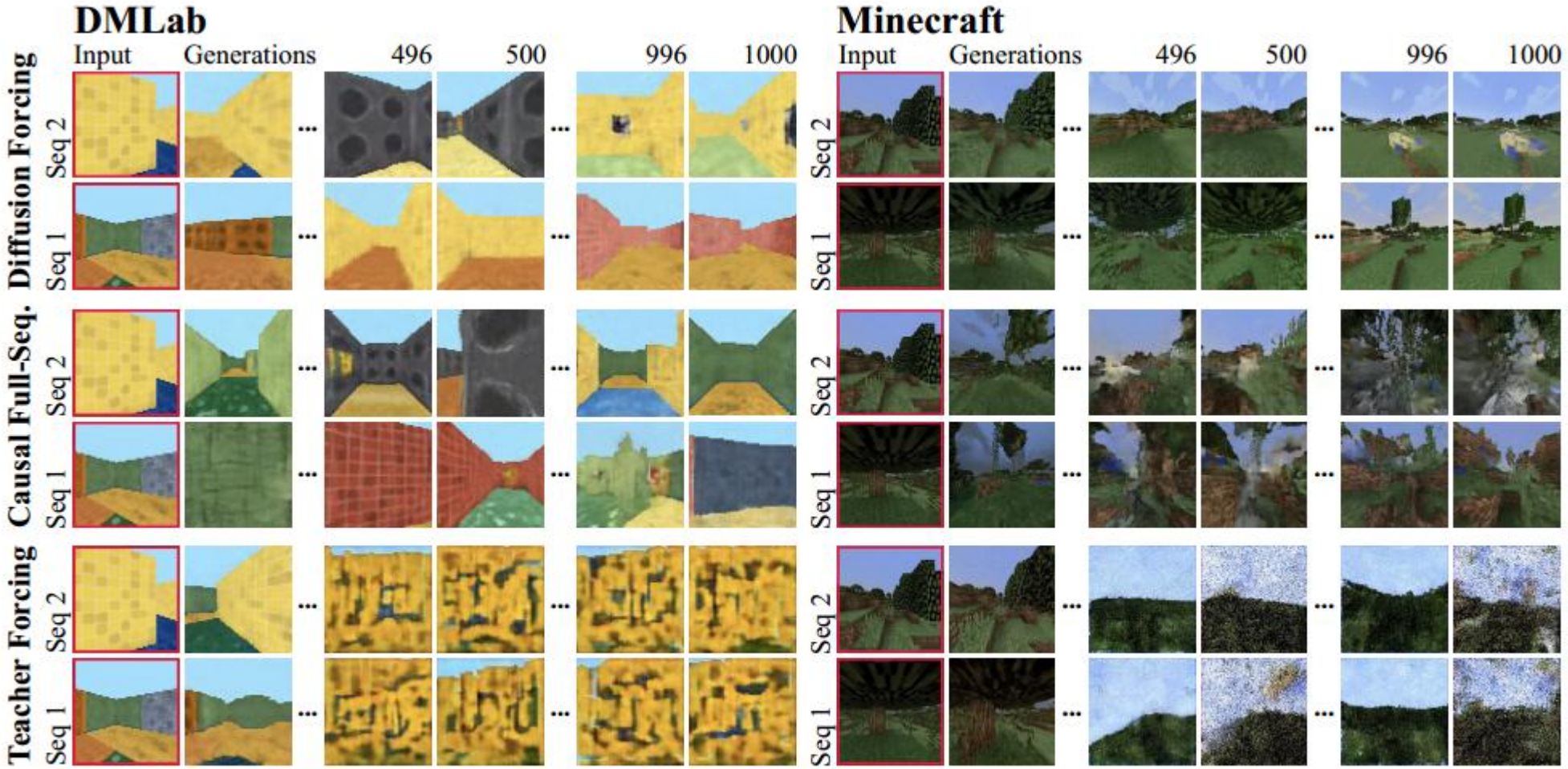
$$[\mathbf{x}_1^K, \mathbf{x}_2^K, \mathbf{x}_3^K]^\top$$
$$[\mathbf{x}_1^{K//2}, \mathbf{x}_2^K, \mathbf{x}_3^K]^\top,$$
$$[\mathbf{x}_1^0, \mathbf{x}_2^K, \mathbf{x}_3^K]^\top$$

Use x_1^1 to diffuse x_2

$$[\mathbf{x}_1^1, \mathbf{x}_2^{K//2}, \mathbf{x}_3^K]^\top$$
$$[\mathbf{x}_1^1, \mathbf{x}_2^0, \mathbf{x}_3^K]^\top$$

$$[\mathbf{x}_1^1, \mathbf{x}_2^1, \mathbf{x}_3^{K//2}]^\top,$$
$$[\mathbf{x}_1^1, \mathbf{x}_2^1, \mathbf{x}_3^0]^\top.$$

Experiment on Video Generation



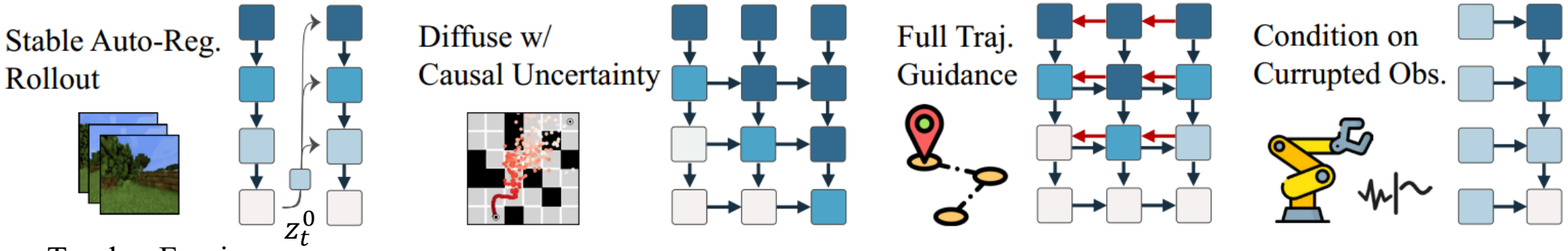
1000 frames

Full SeqDiffusion：frame-to-frame discontinuity

Figure 3: **Video Generation.** Among tested methods, Diffusion Forcing generations are uniquely temporally consistent and do not diverge even when rolling out well past the training horizon. Please see the project website for video results.

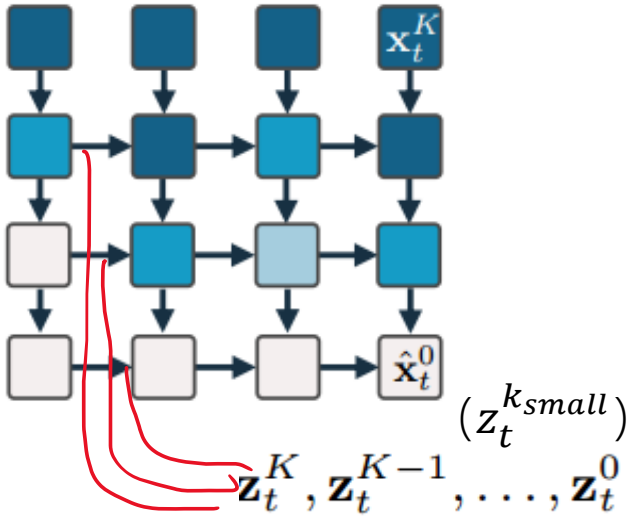
New Capabilities in Sequence Generation

We now explain the new capabilities this flexible sampling paradigm has to offer.



Teacher Forcing

Diffusion Forcing



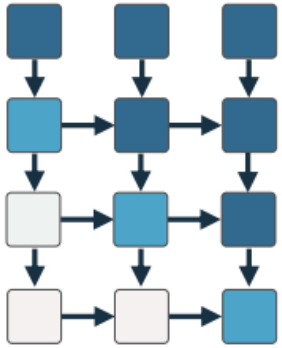
1.Stabilizing autoregressive generation（Intuiton）

——“加噪”能稳定长时序?

- 用$z_t^{k_{small}}$作为latant，而不是取z_t^0，模型从头到尾都在“轻微噪声的 latent 空间”里滚动
- 而训练过程中，模型就已经看到过不同种“带噪状态”组合。
- 在推理过程中：用训练时见过的噪声状态，来作为 autoregressive rollout 的锚点，从而阻断误差级联。

Causal Uncertainty——Diffusion Forcing for Planning

Diffuse w/
Causal Uncertainty



$[\mathbf{x}_1^K, \mathbf{x}_2^K, \mathbf{x}_3^K]^\top$

$[\mathbf{x}_1^{K//2}, \mathbf{x}_2^K, \mathbf{x}_3^K]^\top$

$[\mathbf{x}_1^0, \mathbf{x}_2^{K//2}, \mathbf{x}_3^K]^\top$

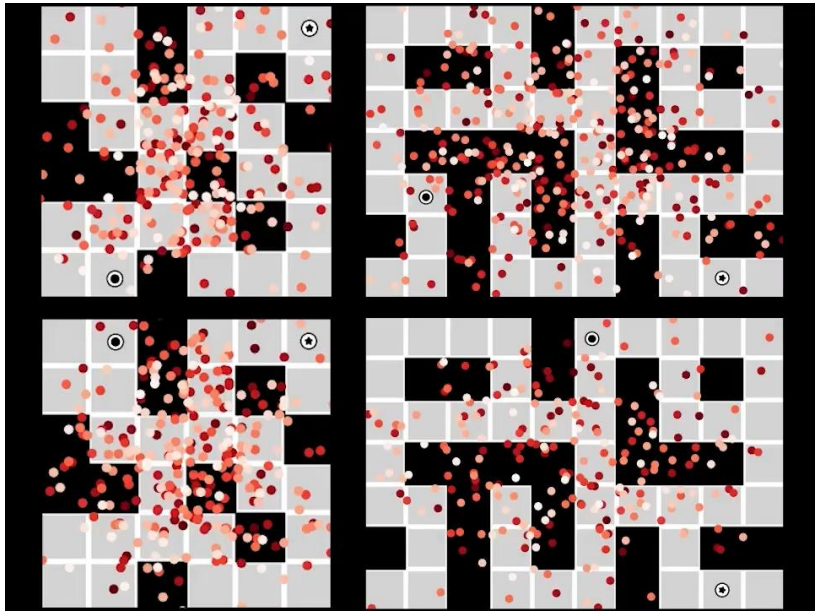
$[\mathbf{x}_1^0, \mathbf{x}_2^0, \mathbf{x}_3^{K//2}]^\top$

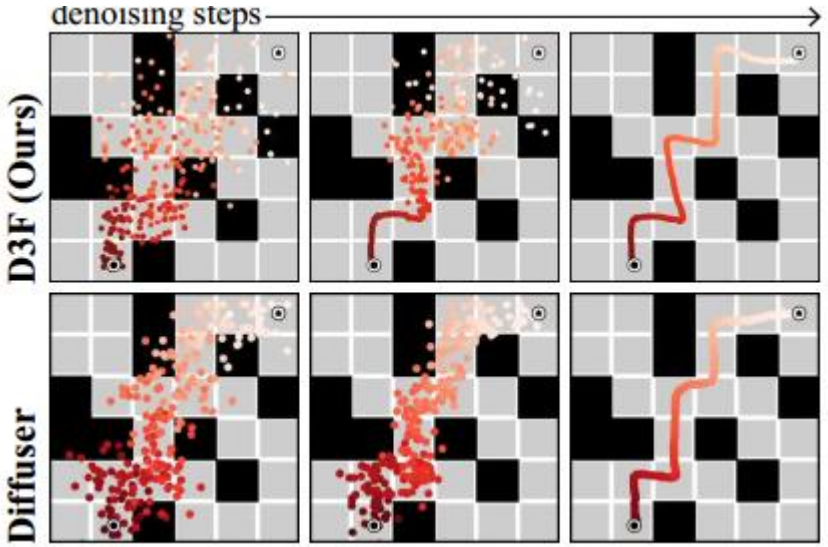
$[\mathbf{x}_1^0, \mathbf{x}_2^0, \mathbf{x}_3^0]^\top$

Masking Strategy—— Independent Noise

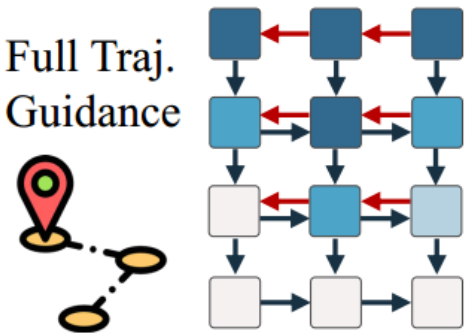
- Earlier elements : Less Noise——Less uncertainty
- Newer elements : Large Noise——Large Uncertainty

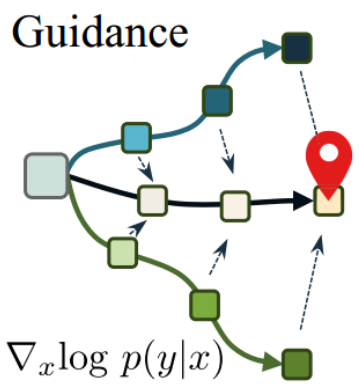
- Full-Seq Diffuion: 所有状态均没有不确定性（一次性生成起始到结尾），所以起步阶段的偏差直接影响结尾
- Diffusion Forcing: 告诉模型，未来是不能完全相信的





Long-horizon Guidance





- AR采样 $x_t \sim p(x_t \mid x_{<t})$

推理内容固定，不能再被优化

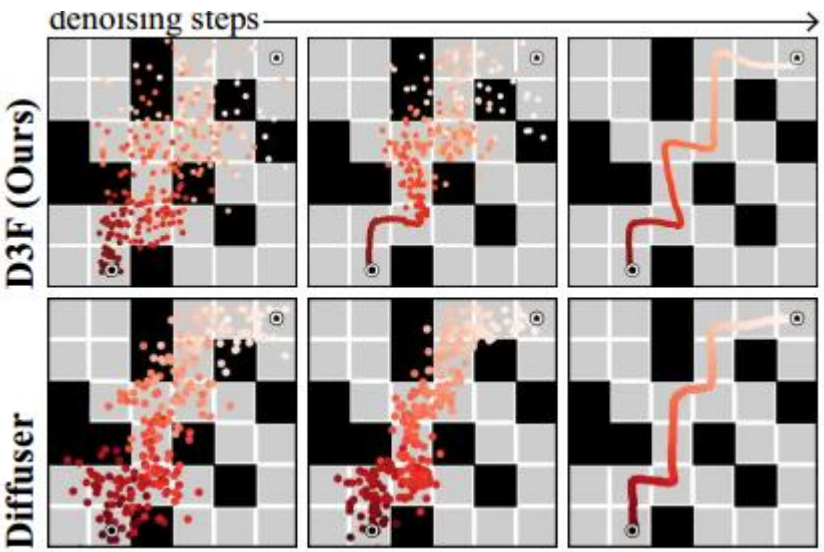
- Full-Seq Diffusion：

模型长度固定、

整个序列（包括过去与未来）一起加噪，因果关系被破坏

- Diffusion Forcing：未来（模糊）的目标影响现在的结果

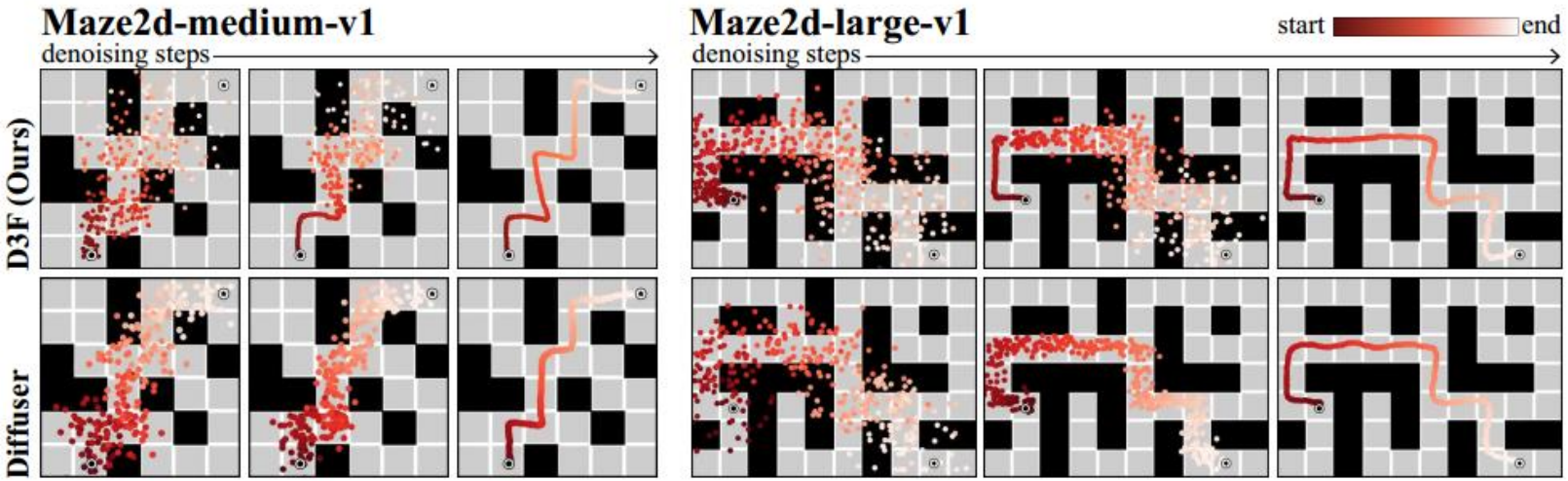
$$\nabla_{x_t^k} \log p(y \mid x_t^k) \longrightarrow \nabla_{x^k} \log c(y|\mathbf{x}^k)$$



路径规划：

将guidance推广：$c(\cdot)$：能量函数 / reward / cost-to-go

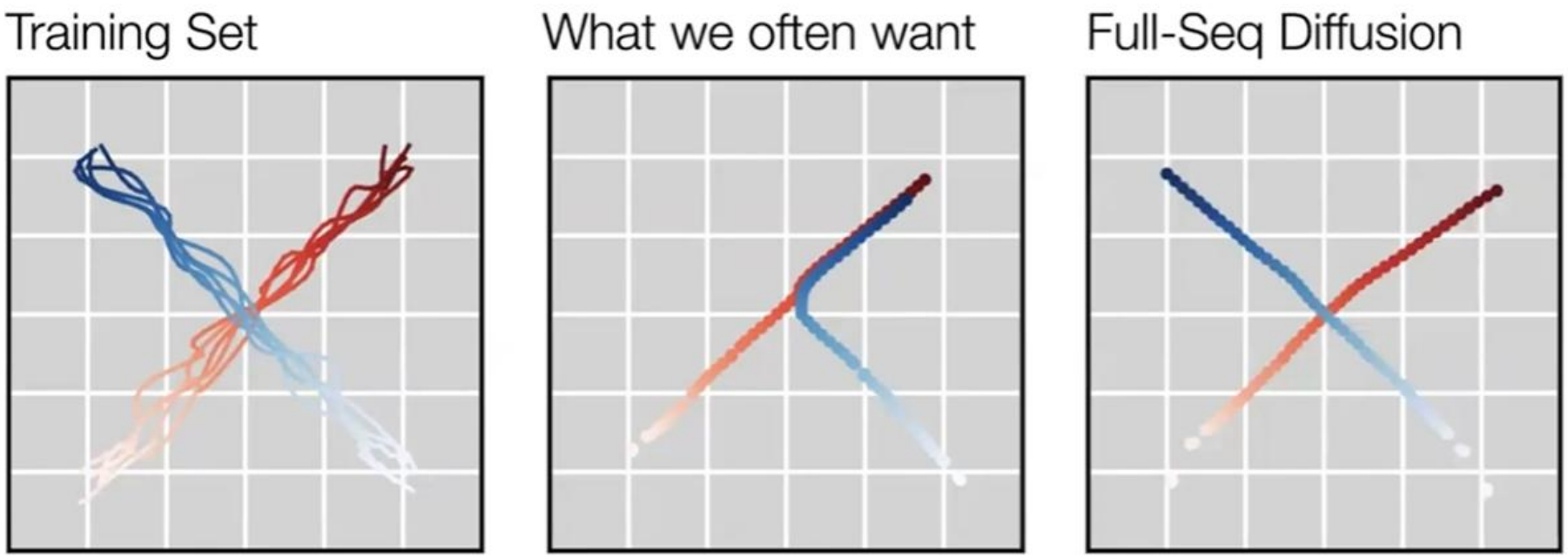
$$c(\mathbf{x}_t^k) = \mathbb{E}\left[\sum_{t'>t} \mathbf{r}'(\mathbf{x}_{t'}^{k_{t'}}) \mid \mathbf{x}_t^k\right]$$



Environment		MPPI	CQL	IQL	Diffuser*	Diffuser w/ diffused action	Ours wo/ MCG	Ours
Maze2D	U-Maze	33.2	5.7	47.4	113.9 ± 3.1	6.3 ± 2.1	110.1 ± 3.9	**116.7 ± 2.0**
Maze2D	Medium	10.2	5.0	34.9	121.5 ± 2.7	13.5±2.3	136.1 ± 10.2	**149.4 ± 7.5**
Maze2D	Large	5.1	12.5	58.6	123.0 ± 6.4	6.3 ±2.1	142.8 ± 5.6	**159.0 ± 2.7**
Single-task Average		16.2	7.7	47.0	119.5	8.7	129.67	**141.7**
Multi2D	U-Maze	41.2	-	24.8	128.9 ± 1.8	32.8±1.7	107.7 ± 4.9	**119.1 ± 4.0**
Multi2D	Medium	15.4	-	12.1	127.2 ± 3.4	22.0±2.7	145.6 ± 6.5	**152.3 ± 9.9**
Multi2D	Large	8.0	-	13.9	132.1 ± 5.8	6.9 ±1.7	129.8 ± 1.5	**167.1 ±2.7**
Multi-task Average		21.5	-	16.9	129.4	20.6	127.7	**146.2**

Table 1: **Diffusion Forcing for Planning.** (**top**) During sampling, Diffusion Forcing allows each time step to be denoised on different noise schedules, enabling us to account for causal uncertainty during guided planning. Diffusion Forcing keeps the far future more uncertain than the near future while Diffuser [37] puts them at the same noise level during sampling. (**bottom**) Quantitatively, Diffusion Forcing achieves the highest average reward across runs. Diffuser fails dramatically when executing the actually generated actions, requiring a hand-crafted PD controller (indicated by the asterisk) and ignoring generated actions.

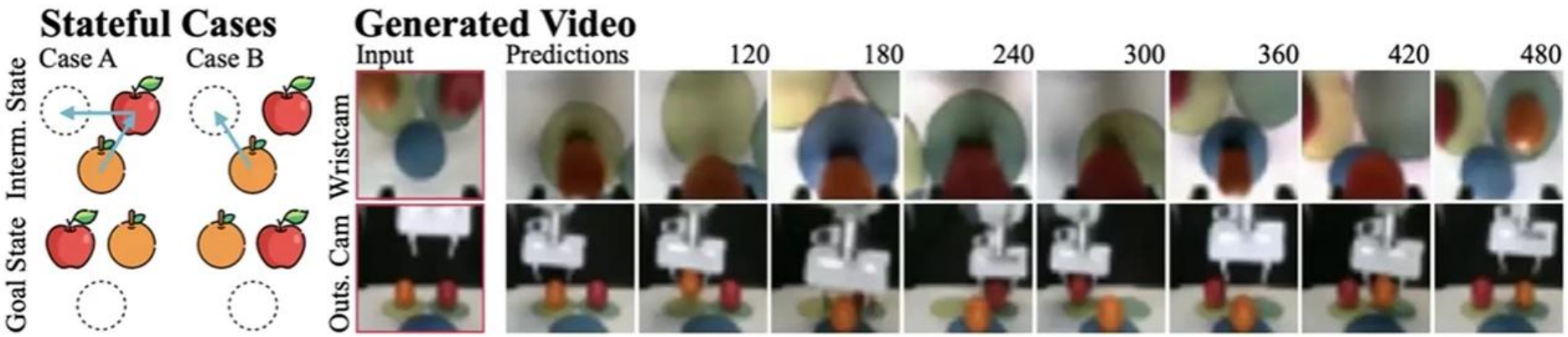
Compositionality

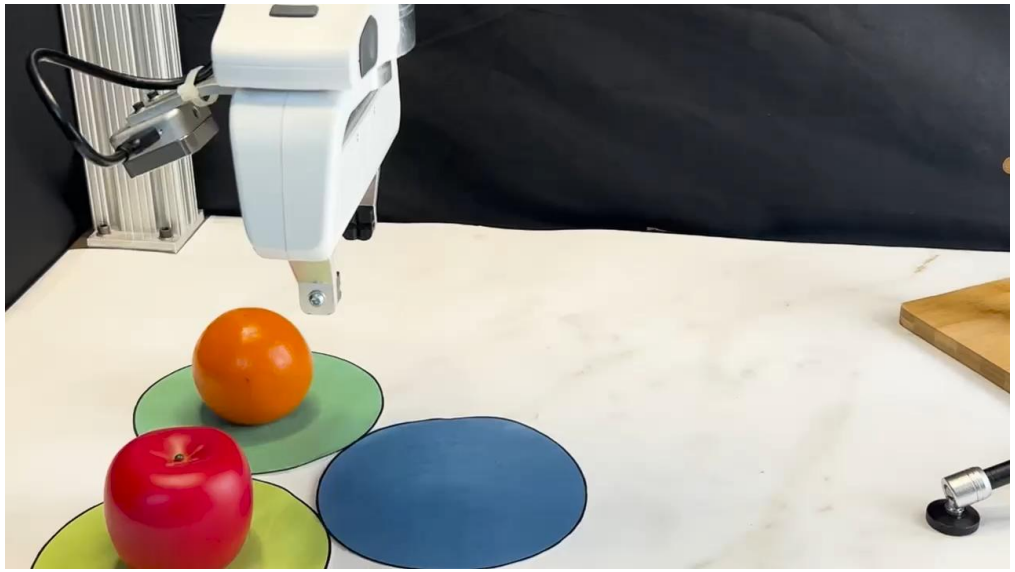


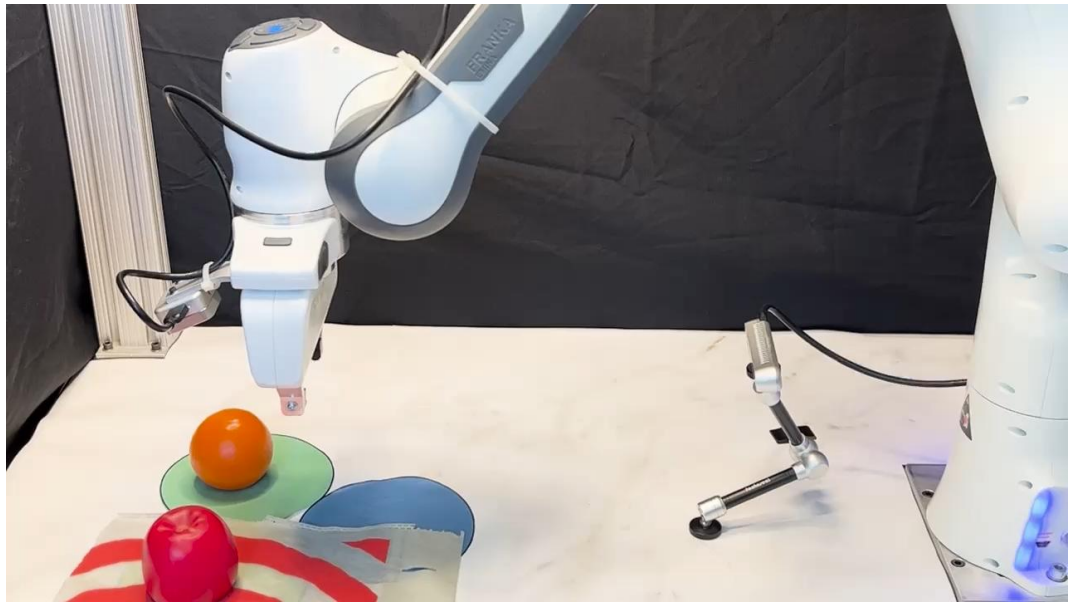
2D轨迹数据集：所有轨迹从一个角开始，结束于对角，形成十字形。

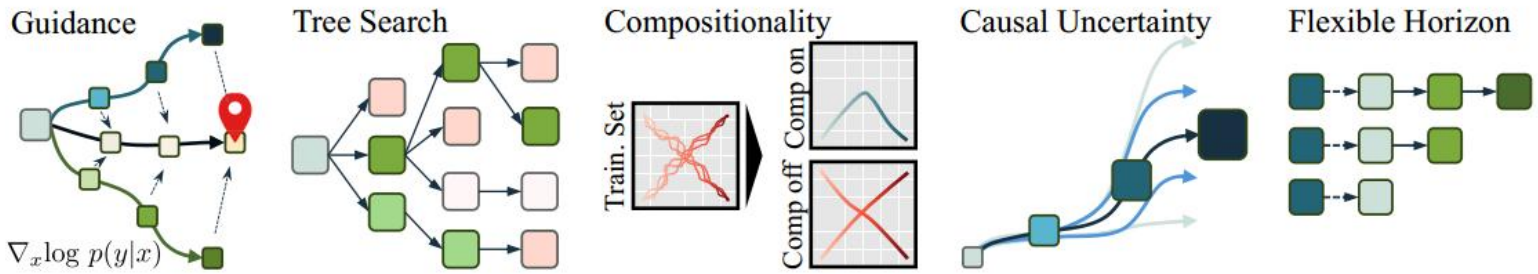
Robotics———“带记忆、可规划、可鲁棒”的序列决策框架

Task: Swap two fruit with the third slot. Non-markov because you need initial state to know target state









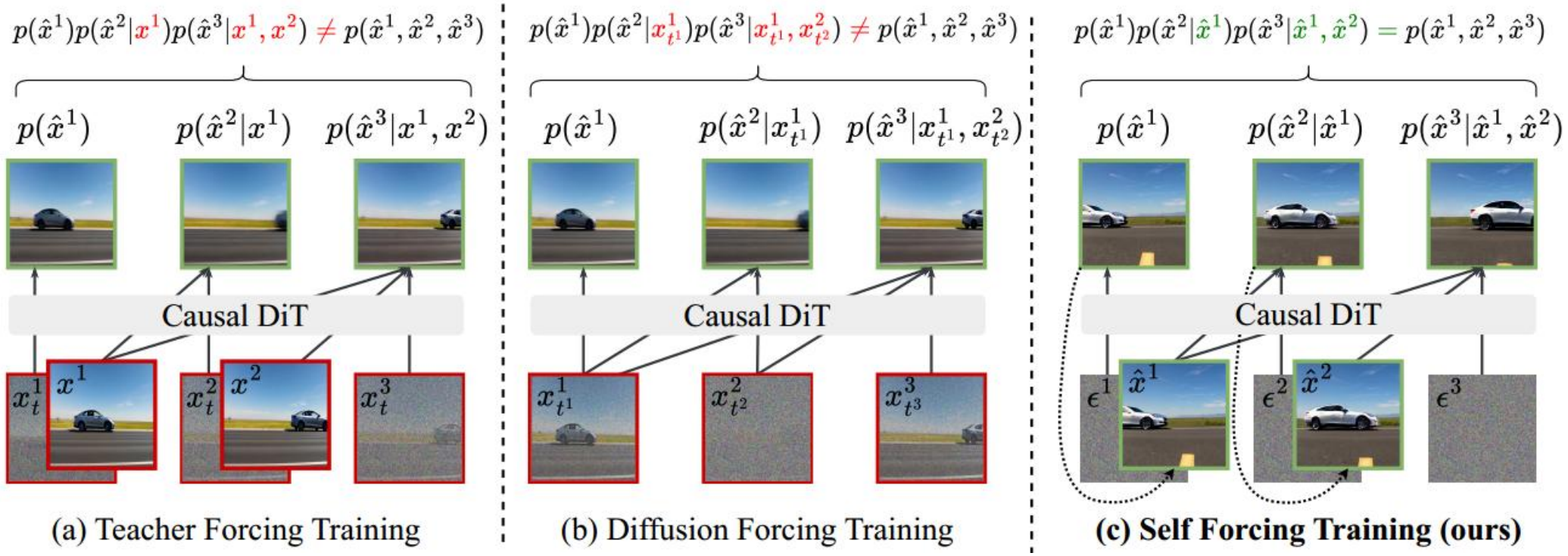
	Guidance	Tree Search	Compositionality	Causal Uncertainty	Flexible Horizon
Teacher Forcing	✗	✓	✓	✓	✓
Full-Seq. Diffusion	✓	✗	✗	✗	✗
Diffusion Forcing	✓	✓	✓	✓	✓

Figure 1: **Diffusion Forcing capabilities.** Today, different applications such as language modeling [6], planning [37], or video generation [32, 70] rely on *either* auto-regressive next-token prediction *or* full-sequence diffusion, according to their respective unique capabilities. The proposed Diffusion Forcing is a novel sequence generative model that enjoys key strengths of both model types.

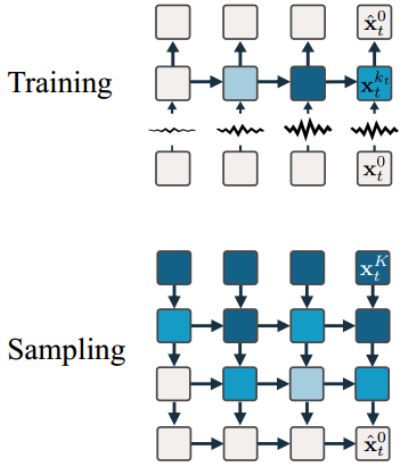
Self Forcing: Bridging the Train-Test Gap in Autoregressive Video Diffusion

Xun Huang[1] **Zhengqi Li**[1] **Guande He**[2] **Mingyuan Zhou**[2] **Eli Shechtman**[1]
[1]Adobe Research [2]The University of Texas at Austin

Training paradigms for AR video diffusion models



- (a) and (b) generate outputs that do not belong to the distribution the model generates during inference.



- Self Forcing：重新设计训练过程，解决 **暴露偏差（exposure bias）**

Self Forcing，在训练阶段中完成扩散模型的推理，使得训练与推理分布一致

Transformer Diffusion

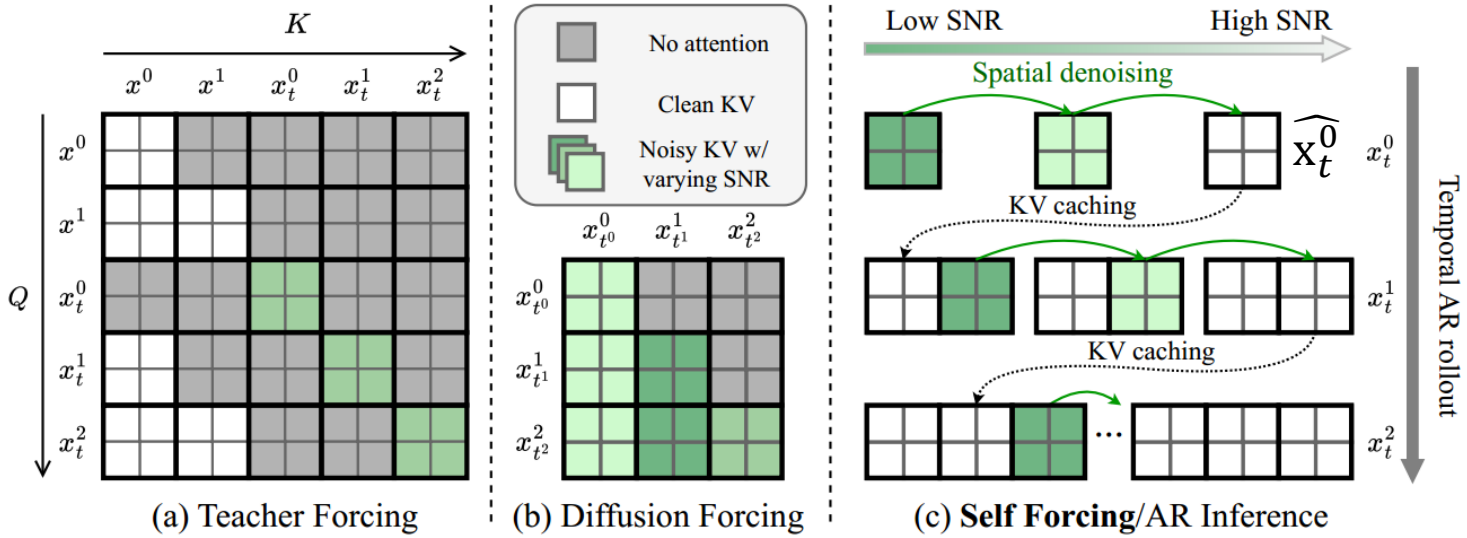


Figure 2: **Attention mask configurations.** Both Teacher Forcing (a) and Diffusion Forcing (b) train the model on the entire video in parallel, enforcing causal dependencies with custom attention masks. In contrast, our Self-Forcing Training (c) mirrors the autoregressive (AR) inference process with KV caching and does not rely on special attention masks. For illustration purposes, we show a scenario where the video contains 3 frames, and each frame consists of 2 tokens.

- 这里放入 KV cache的，是模型预测的结果$\widehat{x_t^0}$，而非真实数据加噪

- generate one chunk of frames rather than a single frame at a time（一次打包生成很多帧——高FPS）

Algorithm 1 Self Forcing Training

Require: Denoise timesteps $\{t_1, \ldots, t_T\}$
Require: Number of video frames N
Require: AR diffusion model G_θ (returns KV embeddings via G_θ^{KV})

```
loop
    Initialize model output X_θ ← []
    Initialize KV cache KV ← []
    Sample s ~ Uniform(1, 2, ..., T)
    for i = 1, ..., N do
        Initialize x^i_{t_T} ~ N(0, I)
        for j = T, ..., s do
            if j = s then
                Enable gradient computation
                Set x̂^i_0 ← G_θ(x^i_{t_j}; t_j, KV)
                X_θ.append(x̂^i_0)
                Disable gradient computation
                Cache kv^i ← G_θ^KV(x̂^i_0; 0, KV)
                KV.append(kv^i)
            else
                Disable gradient computation
                Set x̂^i_0 ← G_θ(x^i_{t_j}; t_j, KV)
                Sample ε ~ N(0, I)
                Set x^i_{t_{j-1}} ← Ψ(x̂^i_0, ε, t_{j-1})
            end if
        end for
    end for
    Update θ via distribution matching loss
end loop
```

Algorithm 2 Autoregressive Diffusion Inference with Rolling KV Cache

Require: KV cache of size L frames
Require: Denoise timesteps $\{t_1, \ldots, t_T\}$
Require: Number of generated frames M
Require: AR diffusion model G_θ (returns KV embeddings via G_θ^{KV})

```
Initialize model output X_θ ← []
Initialize KV cache KV ← []
for i = 1, ..., M do
    Initialize x^i_{t_T} ~ N(0, I)
    for j = T, ..., 1 do
        Set x̂^i_0 ← G_θ(x^i_{t_j}; t_j, KV)
        if j = 1 then
            X_θ.append(x̂^i_0)
            Cache kv^i ← G_θ^KV(x̂^i_0; 0, KV)
            if |KV| = L then
                KV.pop(0)        ▷ Cache eviction
            end if
            KV.append(kv^i)
        else
            Sample ε ~ N(0, I)
            Set x^i_{t_{j-1}} ← Ψ(x̂^i_0, ε, t_{j-1})
        end if
    end for
end for
return X_θ
```

- Train和sample的算法结构相近

- 计算量问题：

（1）训练过程主要耗时集中在扩散步的循环当中；作者选择few-step models：T=4

（2）如果反向传播要从$\widehat{x_t^0}$追溯回$\widehat{x_t^T}$，显存炸

梯度截断方案：

训练时，随机选择一个s∈{1,2,⋯,T}，只扩散到s步就停止

好处：能够学习任意噪声等级的扩散（Diffusion Forcing）

平均角度来看，所有step都能被训练到

$\widehat{x_t^0}$

(x_t5 → f_t5 → x_t4 →) f_t4 → x_t3 (~~→ f_t3 → x_t2~~) → (stop)

只在这里算 loss + 反传，更低噪声不再计算

Holistic Distribution Matching Loss

Algorithm 1 Self Forcing Training

Require: Denoise timesteps $\{t_1, \ldots, t_T\}$
Require: Number of video frames N
Require: AR diffusion model G_θ (returns KV embeddings via G_θ^{KV})

```
loop
    Initialize model output X_θ ← []
    Initialize KV cache KV ← []
    Sample s ~ Uniform(1, 2, ..., T)
    for i = 1, ..., N do
        Initialize x^i_{t_T} ~ N(0, I)
        for j = T, ..., s do
            if j = s then
                Enable gradient computation
                Set x̂^i_0 ← G_θ(x^i_{t_j}; t_j, KV)
                X_θ.append(x̂^i_0)
                Disable gradient computation
                Cache kv^i ← G^KV_θ(x̂^i_0; 0, KV)
                KV.append(kv^i)
            else
                Disable gradient computation
                Set x̂^i_0 ← G_θ(x^i_{t_j}; t_j, KV)
                Sample ε ~ N(0, I)
                Set x^i_{t_{j-1}} ← Ψ(x̂^i_0, ε, t_{j-1})
            end if
        end for
    end for
    Update θ via distribution matching loss
end loop
```

- 我们不再只对“单帧 / 单步”做扩散回归，而是直接在“整段视频层面”，对模型生成的视频分布和真实视频分布做整体对齐。

  需要使用$\widehat{x_t^0}$计算损失而不是ϵ

$$D(p_{\text{data}}(x^{1:N}) \| p_\theta(x^{1:N}))$$

TF/DF frame-wise mean squared error (MSE)

——Diffusion Forcing： $L = \text{MSELoss}([\hat{\epsilon}_1, \ldots, \hat{\epsilon}_n], [\epsilon_1, \ldots, \epsilon_n])$

三种 distribution matching loss：

- Distribution Matching Distillation (DMD)： $\mathbb{E}_t[D_{\mathrm{KL}}(p_{\theta,t} \| p_{\text{data},t})]$
- Score Identity Distillation (SiD)： $\mathbb{E}_{t,p_{\theta,t}}[\|\nabla \log p_{\theta,t} - \nabla \log p_{\text{data},t}\|^2]$.
- Generative Adversarial Networks (GANs)

Long Video Generation with Rolling KV Cache

Algorithm 2 Autoregressive Diffusion Inference with Rolling KV Cache

Require: KV cache of size L frames
Require: Denoise timesteps $\{t_1, \ldots, t_T\}$
Require: Number of generated frames M
Require: AR diffusion model G_θ (returns KV embeddings via G_θ^{KV})

```
Initialize model output X_θ ← []
Initialize KV cache KV ← []
for i = 1, ..., M do
    Initialize x^i_{t_T} ~ N(0, I)
    for j = T, ..., 1 do
        Set x̂^i_0 ← G_θ(x^i_{t_j}; t_j, KV)
        if j = 1 then
            X_θ.append(x̂^i_0)
            Cache kv^i ← G^KV_θ(x̂^i_0; 0, KV)
            if |KV| = L then
                KV.pop(0)        ▷ Cache eviction
            end if
            KV.append(kv^i)
        else
            Sample ε ~ N(0, I)
            Set x^i_{t_{j-1}} ← Ψ(x̂^i_0, ε, t_{j-1})
        end if
    end for
end for
return X_θ
```

- 普通 transformer：对每一帧的基础计算量为L^2 共T个时间步
- 基于滑动窗口的KV cache 的transformer，在移动窗口后，重新计算第一帧时，需要完整计算一次transformer
- Self-forcing：将KV cache加入滚动机制（pop掉末尾的值）

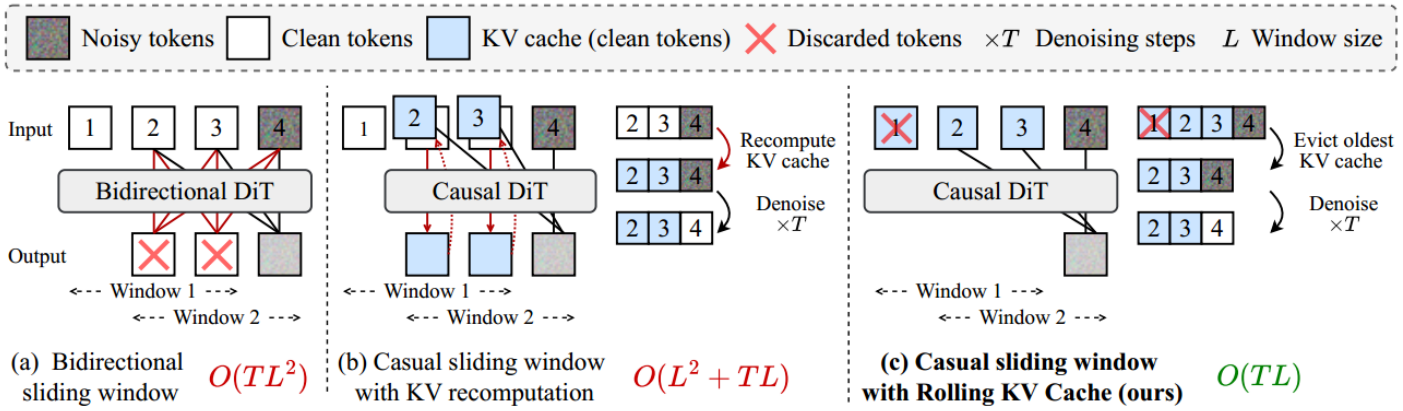


Table 1: **Comparison with relevant baselines.** We compare Self Forcing with representative open-source video generation models of similar parameter sizes and resolutions.

Model	#Params	Resolution	Throughput (FPS) ↑	Latency (s) ↓	Evaluation scores ↑		
					Total Score	Quality Score	Semantic Score
Diffusion models							
LTX-Video [24]	1.9B	768×512	8.98	13.5	80.00	82.30	70.79
Wan2.1 [83]	1.3B	832×480	0.78	103	84.26	**85.30**	80.09
Chunk-wise autoregressive models							
SkyReels-V2 [10]	1.3B	960×540	0.49	112	82.67	84.70	74.53
MAGI-1 [69]	4.5B	832×480	0.19	282	79.18	82.04	67.74
CausVid [100]*	1.3B	832×480	**17.0**	0.69	81.20	84.05	69.80
Self Forcing (Ours, chunk-wise)	1.3B	832×480	**17.0**	0.69	**84.31**	85.07	**81.28**
Autoregressive models†							
NOVA [13]	0.6B	768×480	0.88	4.1	80.12	80.39	79.05
Pyramid Flow [33]	2B	640×384	6.7	2.5	81.72	84.74	69.62
Self Forcing (Ours, frame-wise)	1.3B	832×480	8.9	**0.45**	84.26	85.25	80.30

* We compare with the official implementation of CausVid that uses the same base model (Wan-1.3B).
† The distinction of AR/non-AR applies to the temporal dimension.

- 以chuck为单位一个token生成很多帧——高FPS

Table 2: **Ablation study.** We conduct controlled ablation studies comparing different training paradigms and distribution matching objectives under our training setup across chunk-wise (left) and frame-wise (right) AR models. Self Forcing works well with all different distribution matching objectives and consistently outperforms alternative training approaches.

Chunk-wise AR	Evaluation scores ↑		
	Total Score	Quality Score	Semantic Score
Many (50×2)-step models			
Diffusion Forcing (DF)	82.95	83.66	80.09
Teacher Forcing (TF)	83.58	84.34	80.52
Few (4)-step models			
DF + DMD	82.76	83.49	79.85
TF + DMD	82.32	82.73	80.67
Self Forcing (Ours, DMD)	**84.31**	85.07	**81.28**
Self Forcing (Ours, SiD)	84.07	**85.52**	78.24
Self Forcing (Ours, GAN)	83.88	85.06	79.16

Frame-wise AR	Evaluation scores ↑		
	Total Score	Quality Score	Semantic Score
Many (50×2)-step models			
Diffusion Forcing (DF)	77.24	79.72	67.33
Teacher Forcing (TF)	80.34	81.34	76.34
Few (4)-step models			
DF + DMD	80.56	81.02	78.71
TF + DMD	78.12	79.62	72.11
Self Forcing (Ours, DMD)	**84.26**	**85.25**	**80.30**
Self Forcing (Ours, SiD)	83.54	84.71	78.86
Self Forcing (Ours, GAN)	83.27	84.57	78.08

- 在 diffusion step=4时，在chuck-wise及frame-wise的效果比DF/TF更好——将训练分布与推理分布保持一致

计算效率——diffusions step循环十分耗时

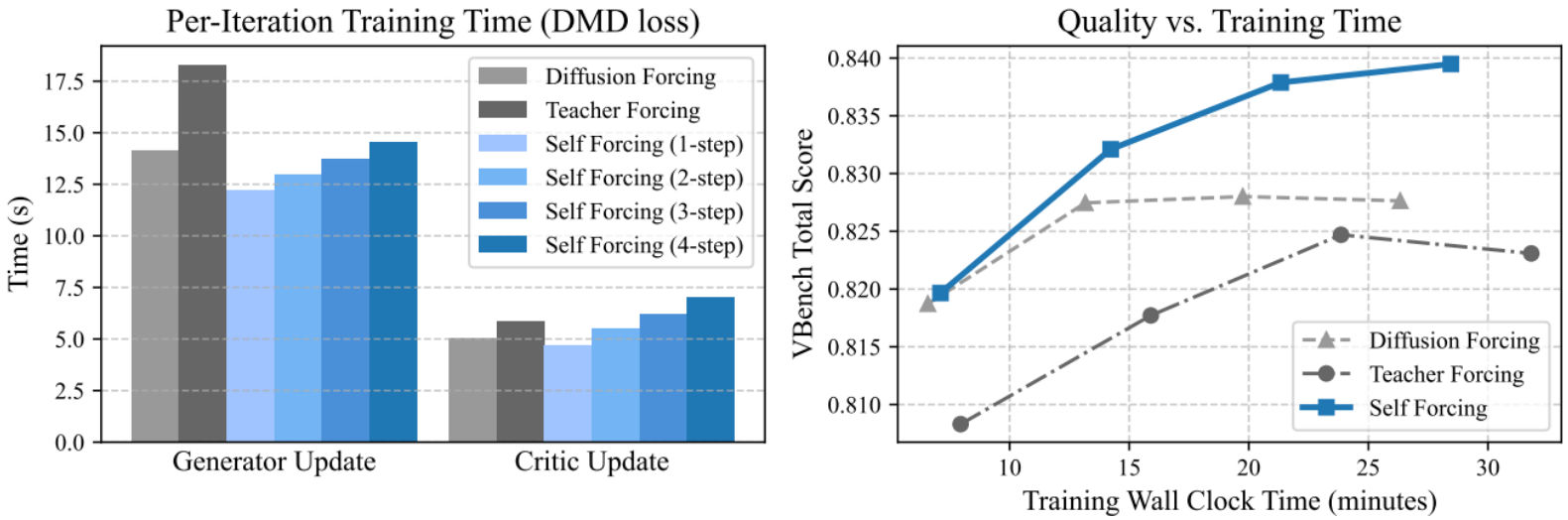


Figure 6: **Training efficiency comparison.** *Left:* Per-iteration time across different chunk-wise, few-step autoregressive video diffusion training algorithms (using DMD as the distribution matching objective). *Right:* Video quality (VBench score) vs. wall clock training time.

- Step越多，耗时越多，但和DF/TF差不多

——使用了KV cache机制、仅一个step进行梯度回传

- Self Forcing是在Diffusion forcing及Teacher Forcing的基础上进行优化，在训练过程中引入Diffusion过程，保证train和inference的结果分布一致

- 并通过若干方法，让training更加高效：
  - Chuck frame
  - 梯度截断
  - Rolling KV cache 不许移动window来重新计算KV cache
  - 扩散步骤T不能大，T=4

- 缺点：接近Teacher Forcing，一次只能生成一个确定的结果，无法用未来结果做引导